

Topic: Prediction of super-popularity in Articles before publication for recommendation systems

Authors: Dovydas Venclovas 15516822, Ignas Jasinskis 15381536, Radin Azadmanesh 15486397 (Group 3)

Abstract: With the ever-increasing availability of media in different sources and formats, the battle for consumer attention is at its highest. Often, users will only look at content for a handful of seconds before their attention moves elsewhere. In this project, we intend to implement machine learning models to predict the most popular news items on any given day to be presented on the front page of news outlets. We implemented three different models under four different conditions (a total of 12 models). We score these models with a customised scoring function we have created. The best model is the baseline XGBoost with a precision score of 0.28 on the customised test set.

Introduction

The fight for the attention of online users is at its peak. Today, consumers of media have shorter attention spans and more content to choose from, resulting in a battle for impressions. The focus of these consumers on each product typically lasts only a few seconds. Therefore, for media outlets and content creators, it is crucial to make a great first impression, whether through an attention-grabbing hook or strategic content placement.

Another method that can help media outlets, especially news sources, to get noticed is to present their best work first. This has historically been the case, with newspapers presenting their most important articles on the front page to increase interactions or news channels presenting their breaking news at the beginning of the show. Detecting your most important article of the day is, therefore, an important task, historically left to the chief editor or the executive producer. In this project, we try to automate this process using machine learning models to predict the most popular articles.

The prediction of the popularity of an article is generally separated into two categories: predicting before publication and after publication. Naturally, prediction after publication can be implemented with user activities, giving extra insight into the probable popularity of new articles. However, many tasks must also be completed before the publication of news articles. The task we have at hand falls into the second category, as media outlets must present their best work at the most visible locations at the moment of publication.

To this end, we implement our models using the Mashable news article database, provided by the UCI machine learning repository (Fernandes, Vinagre, & Cortez, 2015). Our goal is to find extremely popular news articles that drive up impressions and interactions, as well as visits to other pages linked to these articles. These articles are good candidates to be advertised or placed on the front pages of news outlets.

Methodology

We are looking not only for popular articles, but extremely popular articles with the potential of grabbing consumers' attention and increasing interactions. To this end, we take the top 10% of popular articles as the super-popular class and the rest as the unpopular. We use both classification and regression models to predict the most popular articles. As this data split creates an imbalance between the two categories, we try two different balancing techniques with the baseline model for classification tasks. We generally presume that, due to the high imbalance, the baseline model will do worse than the two balancing techniques.

For regression, we implement different strategies to get good predictions. As regression models do not provide prediction confidence levels, we use the predictions with the highest number of shares as the most popular predictions.

An argument for resampling versus regression

Due to the fact that the target labels received a split of 90/10, it is clear that the classification problem for this dataset is extremely unbalanced. For this reason, it was decided to use both undersampling and oversampling as a means to strengthen overall model performance. Alternatively, the problem of class imbalance can be avoided by simply not having any classes, and therefore making this a regression problem.

Data preparation and splits

For our target variable, we made one version for classifying tasks and one version for regression. For classification, the rows with the highest 10% of shares were labelled 1, and the other 90% with 0. For regression, the shares were log-transformed since the distribution was very right-tail skewed [Figure 1] and then left as a numerical feature. All feature variables except 'url' and 'timedelta' were used for training the models.

We continued by sorting the dataset by timedelta in descending order so that the oldest articles appear first. Then we split it **without randomising**, allocating the first 85% to training and 15% to test sets. Our line of reasoning for this is that the collected articles span roughly 2 years, so a 15% split gives us approximately three and a half months. We then split the test set into 100 even, unrandomized sets, each containing 60 rows. With this modelling, each training subset represents one day, essentially simulating how our model would perform the first 100 days after implementation. The objective is to get daily predictions, as article outlets often have to present new articles daily.

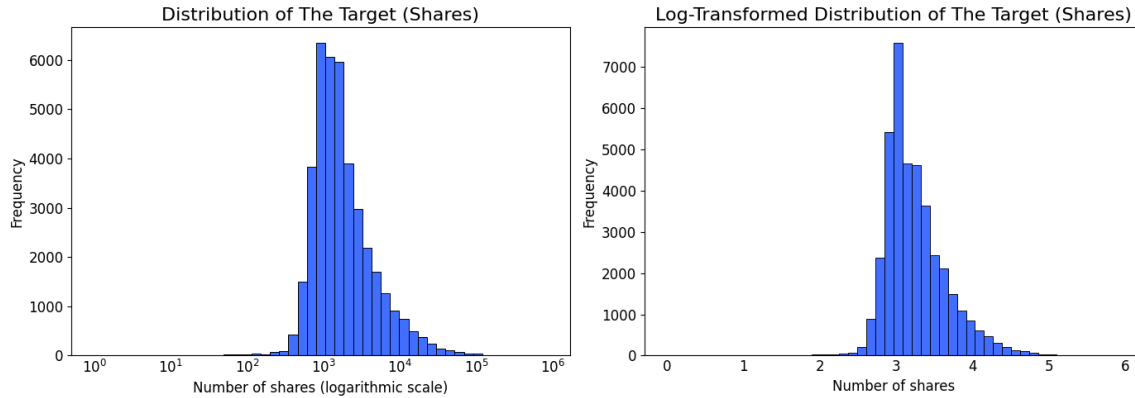


Figure 1. *Effects of log-transforming the target variable. Note the skewness due to a logarithmic axis in the first graph and how this effect is mitigated.*

Test scores

As mentioned earlier, the test set contains around 100 days' worth of the most recent articles in the data set. This set is split into 100 subsets, each representing 1 day with approximately 60 articles each. The expected number of super-popular articles in each of these splits is therefore 6.

To test our models, the model has to make predictions for each of these 100 data sets separately. Each day, the model returns the three most confident predictions. In classification, this is given by classification probabilities, and in regression, this is given by predictions with the highest predicted number of shares. Then the precision of these articles being super popular is calculated for each day (a result between 0, $\frac{1}{3}$, $\frac{2}{3}$, and 1). The final test score is the average of the precisions in these 100 days.

Consistency across models with cross-validation

To fairly validate how different models score, some ground rules were established.

1. For each model, 4 different pipelines were built. Each pipeline had i) a baseline classification model, ii) an undersampled classification model, iii) an undersampled classification model with additional oversampling using SMOTE, iv) a regression model.
2. Most of the optimizable parameters were omitted. Each model had a parameter grid with 3 parameters that we decided were the most impactful for the overall performance. All models were cross-validated using a time series split with 3 splits to reflect our overall business objective. We selected a generally low number of splits for computational and time purposes.

3. All cross-validation was done under our own scoring functions. This scoring function sets the threshold of positive class prediction to the 90th percentile of positive prediction probability, increasing the precision of the predictions as much as possible. By not fixing the precision threshold, we allow the models to always make positive predictions, but only score their most confident prediction. We use the precision of these most confident, positively classified articles as the cross-validation score.
4. For regression models, we took a different approach. The scoring function for cross-validating regression looks at the precision of the top 10 percentile of samples with the highest predicted number of shares. The final test score is calculated using the precision of predictions with the highest number of shares.

Results and Conclusion

The results from 12 models revealed several consistent patterns about the impact resampling has on the models, relative difference between classifiers and regressions for this objective, as well as differences in models themselves.

Cross-validation performance

The cross-validation (CV) results indicate how well the different models and settings generalize classification and regression models. Across all three algorithms (XGBoost, Random Forest and CatBoost), the classification models consistently but barely outperformed the regression models.

Among all pipelines, Catboost with baseline classification setup achieved the highest average CV score of 0.268, followed by its undersampled version with 0.264. As seen in [Figure 2], XGBoost and Random Forest models achieved slightly lower scores, but the relative differences between models are not significant (XGBoost baseline with 0.262 and with undersampling – 0.255, while Random Forest baseline with 0.261 and undersampled variant with 0.261).

Resampling techniques – undersampling and SMOTE – did not result in improvements in CV scores, and in several cases even reduced the performance of algorithms. This suggests that the original data distribution was likely informative enough. Another reason could be that tree-based ensemble models usually perform better when dealing with imbalanced data in the first place, so by undersampling, we lose information that hinders model performance.

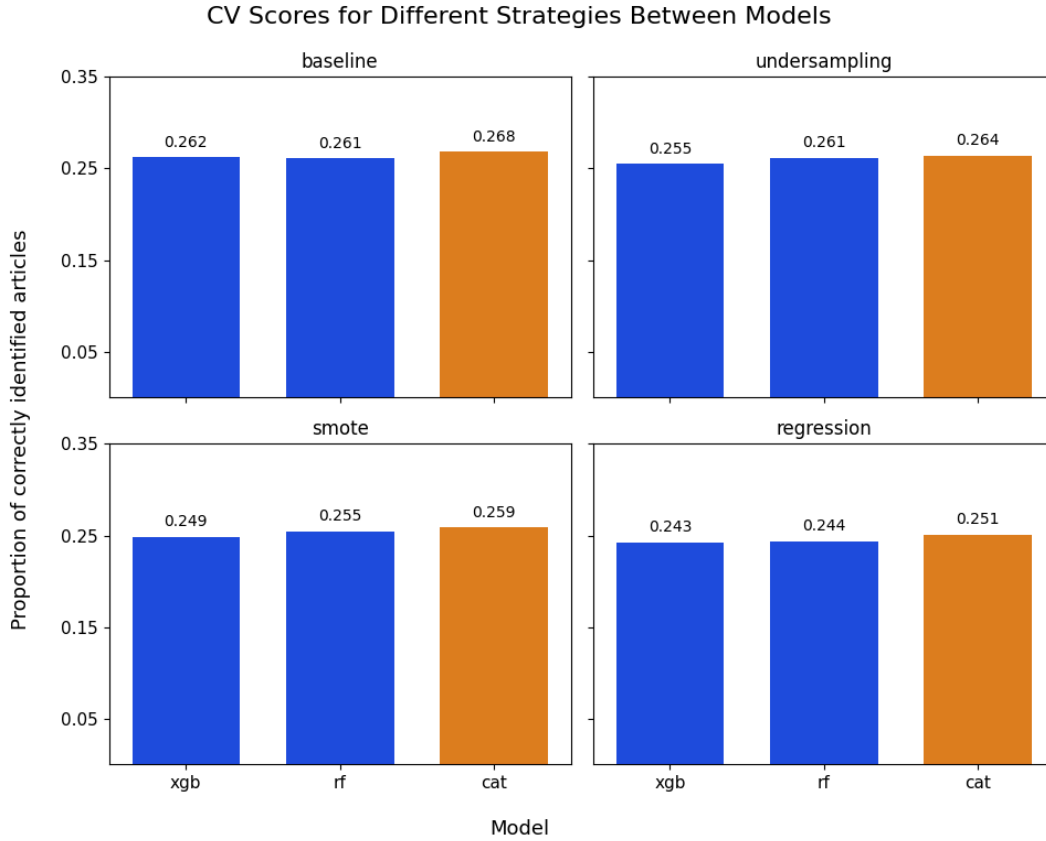


Figure 2. *Cross-Validation (CV) best scores for each model with different data preprocessing strategies.*

Performance in predicting the TOP 3 articles daily

Test scores, calculated using 100 simulated daily predictions, as mentioned in the Test Scores section, provide a realistic estimate of how generalized these models are and will perform in real-life situations. The scores presented in [Figure 3] showcase that the best-performing model on the test set was XGBoost with the baseline classification pipeline, achieving an average daily precision of 0.28¹ when selecting the three most confident articles per day. CatBoost also performed strongly, particularly with the undersampled (0.27) and regression (0.26) pipelines, while Random Forest models displayed the test scores in the range of 0.22 – 0.247, making it the least precise model.

¹ Note that this and all following results are already converted to the same scale as CV scores (from 0 to 1), and not the original best out-of-three scale. Therefore, a score of 0.28 implies 2.8 out of 10 articles would get classified correctly, on average.



Figure 3. *Best test scores for each model with different data preprocessing strategies.*

Due to the fact that test scores are so close to CV scores [Figure 4], we can observe that the generalizability of the model is fairly constant using the tuned parameters. Perhaps better CV results and test scores can be achieved with more varied hyper-tuning, not applied in this research, mainly due to time constraints. Additionally, SMOTE pipelines consistently underperformed in tests, meaning that synthetic data may have introduced patterns that did not and most likely will not persist over time.

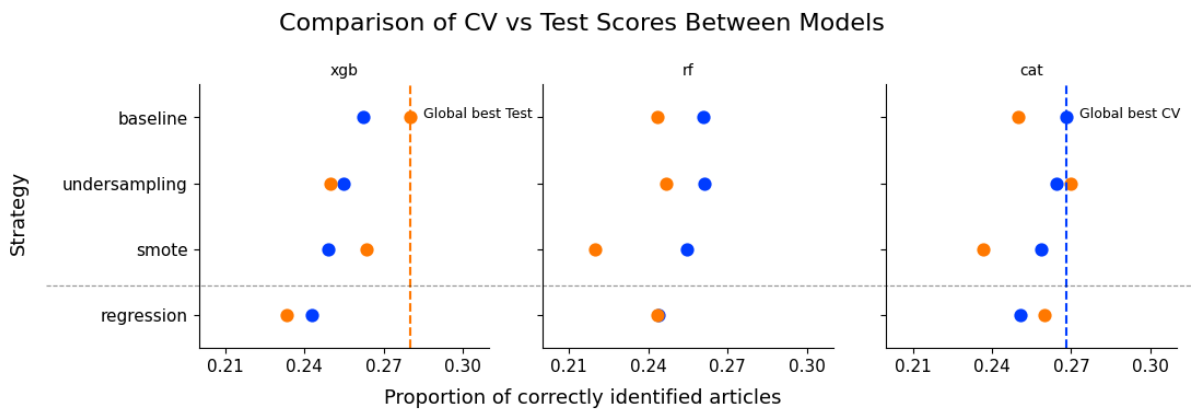


Figure 4. *Comparison of CV and test scores between models. The dashed line indicates that scoring functions slightly vary between classification and regression models and thus should be compared with this distinction in mind.*

Another score that can represent the strength of the models is the proportion of days when the models had at least one correct prediction (out of three). This metric describes the number of days (out of 100) where the model places a super-popular article on the front page, increasing the interactions. As seen in [Figure 5], the baseline XGBoost also works best on this metric, making at least one correct prediction in 63 days out of 100.

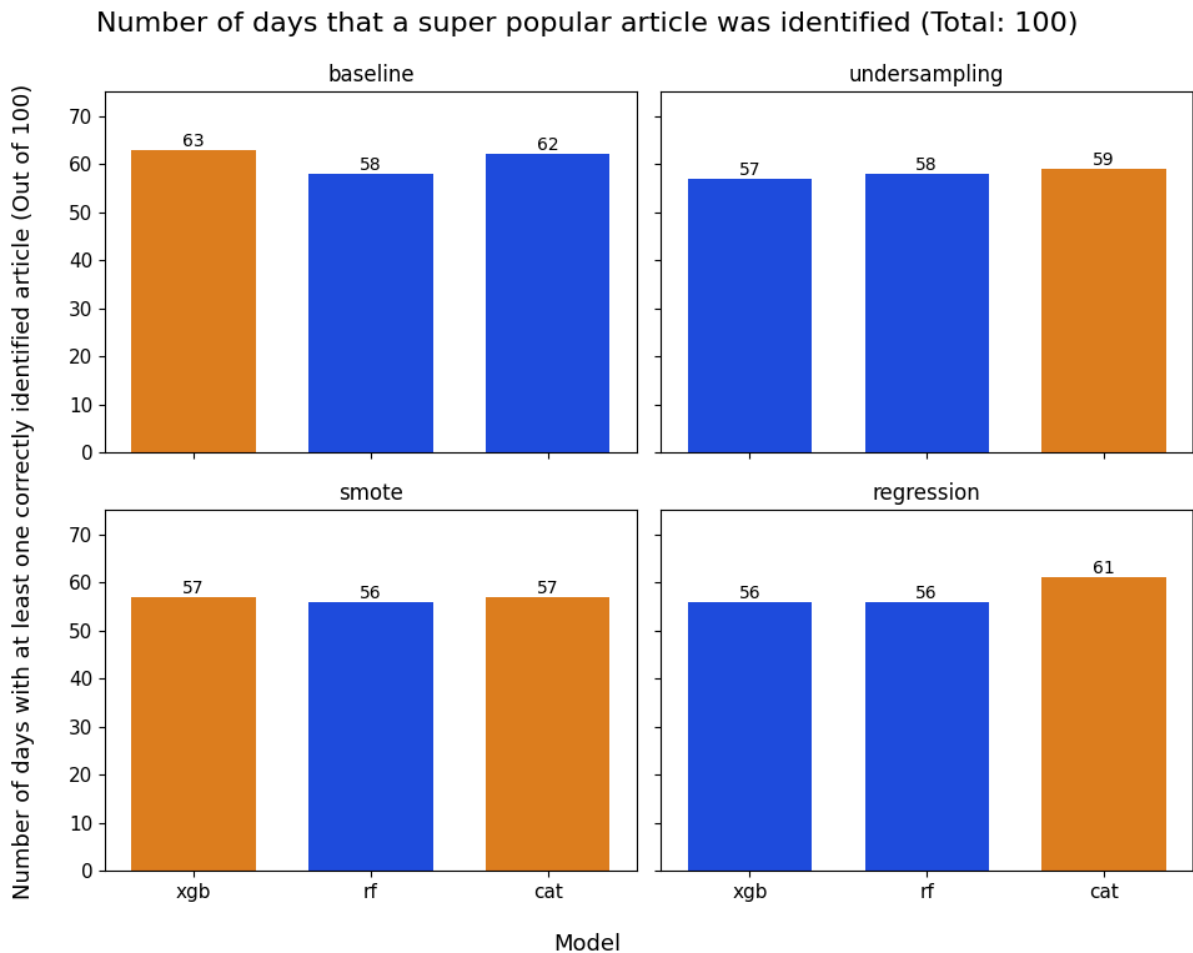


Figure 5. *The number of days on which the model predicted at least one super popular article correctly (out of 100 days).*

Overall

All in all, the results conclude that for the ‘Article Popularity’ dataset, classification models are better suited than regression models for identifying super-popular articles, when comparing precision-focused scores and top-k predictions. In general, resampling techniques such as undersampling and SMOTE did not improve the CV scores (training performance) or the test scores, which is not entirely consistent with our initial predictions. Even though the pipelines’ performances were similar in all settings, the baseline XGBoost model performed the best and achieved the highest test score of 0.28.

Applications to Business

From a business and news outlet standpoint, the results show that even slightly underfit and perhaps overly simplistic machine learning models can bring value. With an average daily precision of 25–28% when predicting the three most popular articles per day, the models with different data preprocessing methods substantially outperform random selection, where expected precision is 10%, based on the class split of 90/10.

Additionally, these models enable news portals to consistently choose around one super-popular article per day at the publication time, increasing shares and impressions – main targets for such business models. Previously mentioned mild underfitting may be even beneficial in this context, as these models prioritizes stability and consistency rather than the most precise predictions, which could come at a cost of higher variance across different days.

Overall, the study demonstrates that machine learning models can increase performance and effectively suggest the right articles to place on the front page, resulting in a balance between stability and increasing value for the business.

References

- K. Fernandes, P. Vinagre and P. Cortez. A Proactive Intelligent Decision Support System for Predicting the Popularity of Online News. Proceedings of the 17th EPIA 2015 - Portuguese Conference on Artificial Intelligence, September, Coimbra, Portugal.